

Introduction

This report summarizes the outcome of the meeting between Kit Armstrong, Tuur Rumes, and Pieter-Jan Maes on 23 July 2026. The meeting focused on translating the concept of reusable Beethovenian compositional “blocks” into a computational framework based on NeuralConstraints (Vassallo, 2025). The resulting proposal centers on a first proof-of-concept implementation of Block #1.

Concept: Beethoven blocks as parameterized compositional procedures

The proposed approach builds on the framework of NeuralConstraints (Vassallo et al., 2025), which combines neural prediction with explicit compositional rules enforced through a constraint solver, rather than relying on a neural network to generate an entire composition autonomously.

Within this framework, the input consists of symbolic musical material, such as a Beethoven sketch or piano reduction. Rather than being transformed directly into an orchestral score, the input is processed by a Beethoven Block: a formal, machine-readable specification of a reusable compositional procedure. A block does not contain fixed musical content. Instead, it defines how musical material should be interpreted and transformed into an orchestral realization.

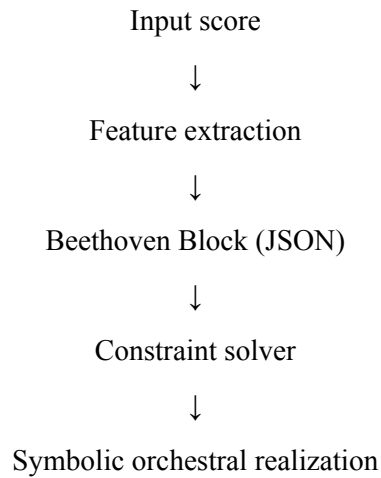
Beethoven Blocks can therefore be understood as parameterized functional transformations. They operate on musical functions—such as melody, harmony, bass, texture, or motivic material—and specify the procedures through which these functions are realized in an orchestral context.

Conceptually, each block answers the question:

« Given this musical material, which musical functions does it contain, and how should these functions be transformed into an orchestral realization according to a characteristic Beethovenian procedure? »

Each block is instantiated as a JSON specification containing the relevant constraints, rules, and parameters. Rather than encoding only low-level restrictions, such as pitch range or prohibited intervals, the JSON captures higher-level compositional procedures.

The novelty of the proposed approach does not simply lie in applying constraint-based generation to orchestration. Instead, it introduces Beethoven Blocks as an intermediate computational layer between symbolic musical input and the NeuralConstraints solver. Each block represents a reusable functional transformation that specifies how a particular type of musical material should be interpreted and realized orchestrally. Different blocks can therefore be selected and parameterized while using the same underlying generative framework.



Block#1

As a next project milestone, we will implement a first Beethoven Block (Block#1) within the NeuralConstraints framework as a proof of concept. This implementation will demonstrate how a reusable Beethovenian compositional procedure can be formalized as a machine-readable JSON specification and integrated into the existing constraint-based generation pipeline.

The initial implementation focuses on Beethoven Block #1 (Melody–Harmony–Bassline realization). Given symbolic input material (e.g., a Beethoven sketch or piano reduction), the system first extracts the musical functions required by the block, such as the principal melody, harmonic progression, and structural bassline. These functions are assigned to three corresponding orchestral layers—Melody, Harmony, and Bassline—each associated with a predefined set of orchestral instruments. Rather than orchestrating the symbolic input directly, each layer applies its own functional transformation rules to realize the extracted musical function orchestrally.

The block itself is encoded as a JSON specification. Layer-specific rules govern how melody, harmony, and bassline are transformed, while shared constraints ensure coordination and balance between layers, and instrument-specific constraints account for factors such as range, playability, and timbral characteristics. Realization parameters—including orchestral density, doubling, independence of inner voices, contrapuntal complexity, timbral richness, dynamic weight, and textural transparency—allow different orchestrations to be generated from the same underlying musical functions. The

resulting rule set is interpreted by the NeuralConstraints solver to generate the final symbolic orchestral realization.

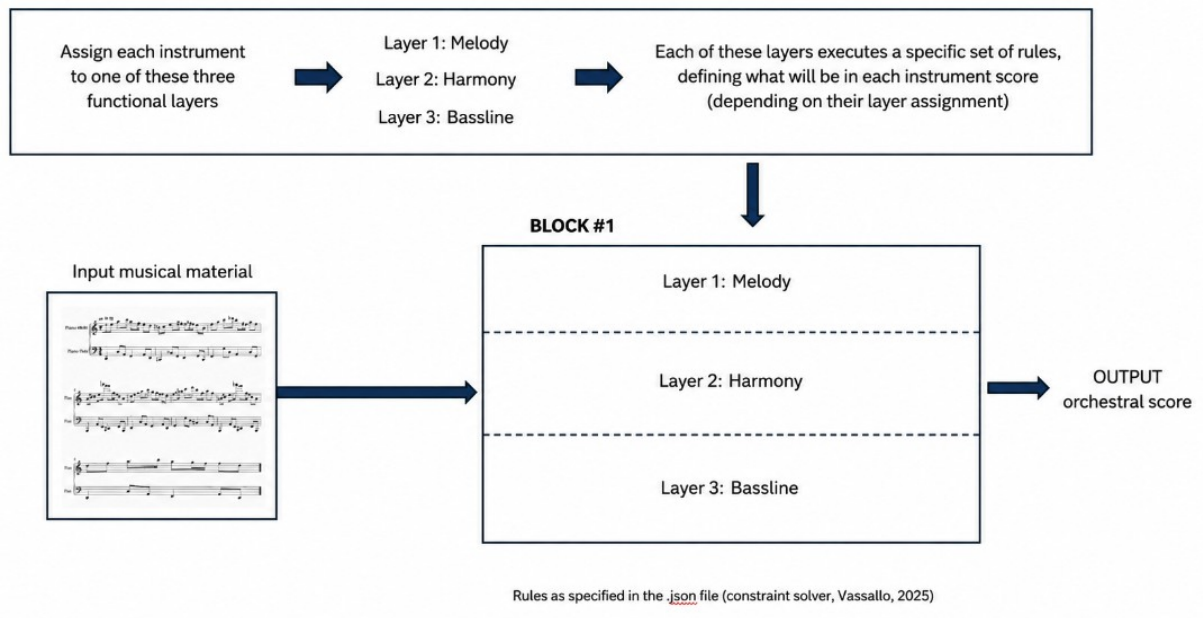
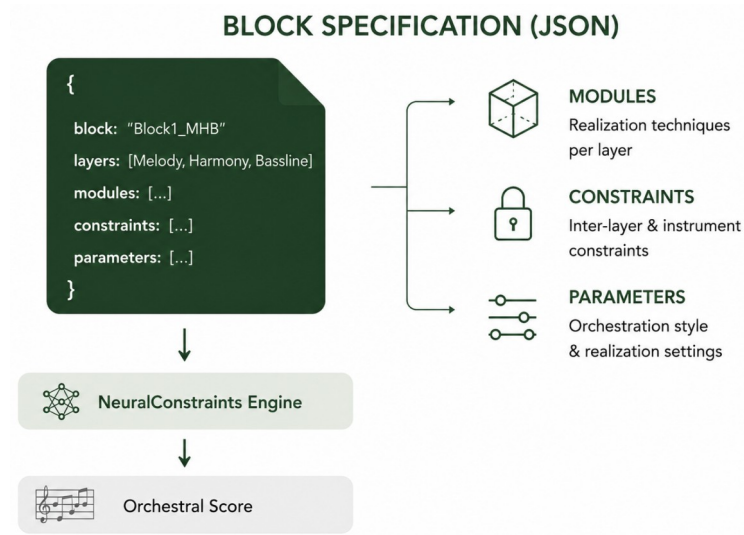


Fig.

Layer	Preserves	Realizes through
Melody	Melodic identity	Instrumental expression
Harmony	Harmonic function	Contrapuntal inner voices
Bassline	Harmonic foundation	Orchestral bass support

Fig. Overview of the different musical layers within Block#1. Each preserves a different musical function, and each has its own family of realization techniques.



LAYER 1 – Melody

The Melody layer does not simply copy the melody from the input material. Instead, it first identifies the principal melodic line and extracts the melodic function that will serve as the basis for orchestral realization. This function preserves the essential musical characteristics of the input—such as melodic contour, rhythmic identity, phrase structure, and structurally important pitches—while allowing the realization to adapt to the assigned instruments. Depending on the orchestration strategy, the realization may include register adaptation, octave displacement, redistribution of the melody across instruments, articulation and dynamic shaping, breathing points for wind instruments, or the omission of ornamental notes.

MELODY LAYER – RULE STEPS

From symbolic input to orchestral melodic realization

Step	Rule	Example
1. Extract	Identify the principal melodic line in the input material and extract the melodic function.	Piano texture → principal melody (melodic function).
2. Preserve	Preserve the essential musical characteristics of the melodic function: melodic contour, rhythmic identity, phrase structure, and structurally important pitches.	Contour, rhythm, phrase boundaries, structurally important notes.
3. Generate	Generate a realizable melodic line for orchestral instruments based on the extracted function.	Melodic line adapted for orchestral realization.
4. Assign & Optimize	Assign the melody to the Melody layer instruments and optimize register, tessitura, playability, and balance within the orchestral context.	Violin I / Flute / Oboe, register optimization, playability, balance.
5. Realize	Realize the melody using orchestration strategies that adapt its expression to the instruments and the orchestral texture.	Orchestration strategies for the melody: • Register adaptation • Octave displacement • Redistribution across instruments • Articulation shaping • Dynamic shaping • Breathing points for wind instruments • Omission of ornamental notes

LAYER 2 – Harmony

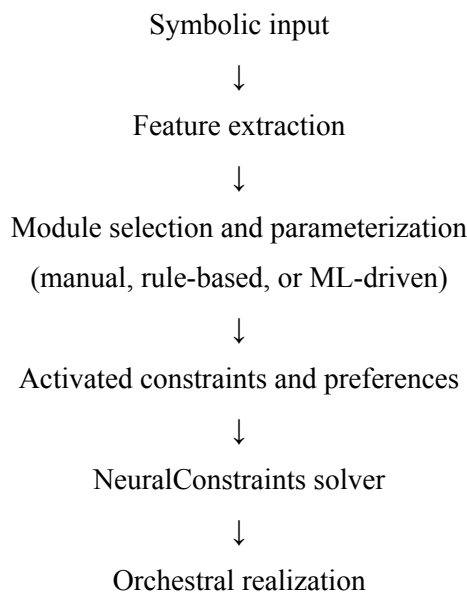
The Harmony layer transforms the harmonic content of the input material into a coherent orchestral harmonic texture. It first extracts the harmonic progression (i.e., the underlying succession of chords or harmonic functions). Based on this, it generates independent inner voices that together realize the harmony. These voices are then assigned to the instruments of the harmony layer—e.g., horn, viola and bassoon—and optimized for smooth voice leading. The realization may employ orchestration principles such as harmonic reduction, sustained harmony, voice redistribution, common-tone retention, contrary motion, register optimization, incomplete chords, or timbre blending, depending on the desired orchestral texture. The result is an orchestral harmony that preserves the harmonic progression and its structural functions while transforming the repeated or pianistic accompaniment into sustained, contrapuntal inner voices that slow down the harmonic rhythm and create a clear, transparent harmonic colour.

HARMONY LAYER – RULE STEPS

From symbolic input to orchestral harmonic realization

Step	Rule	Example
1. Extract	Extract the underlying harmonic progression (chords / harmonic functions) from the symbolic input.	Piano accompaniment → harmonic progression (I–V–vi–IV ...)
2. Preserve	Preserve the harmonic progression, harmonic functions, cadential structure and tonal stability.	Chord sequence, tonic–dominant relationships, harmonic rhythm (at a structural level).
3. Generate	Generate independent contrapuntal inner voices that realize the harmony.	Continuous harmonic voices instead of repeated piano accompaniment.
4. Assign & Optimize	Assign voices to the Harmony layer (e.g. Horn–Viola–Bassoon) and optimize voice leading.	Smooth motion, common–tone retention, register optimization, minimal crossings.
5. Realize	Produce an orchestral harmonic texture.	Orchestration techniques related to harmonic accompaniment that are typical for Beethoven: • Harmonic reduction • Sustained harmony • Technique 1 — Voice redistribution • Technique 2 — Incomplete chords • Technique 3 — Common–tone retention • Technique 4 — Contrary motion • Technique 5 — Independent inner voices • Technique 6 — Register optimization • Technique 8 — Doubling only structurally important notes • Technique 9 — Dynamic hierarchy • Technique 10 — Timbre blending • Technique 11 — Harmonic pedal • Technique 12 — Harmonic anticipation • Technique 13 — Harmonic delay • Technique 14 — Harmonic colour through instrumentation • Technique 15 — Countermelodic accompaniment

In terms of implementation, within each Beethoven Block, orchestration principles could be represented as modular rule and constraint sets in the JSON specification. For the Harmony layer, these include techniques such as harmonic reduction, voice redistribution, common-tone retention, contrary motion, incomplete chords, register optimization, timbre blending, and harmonic pedal. Depending on the implementation, modules may be selected manually, parameterized, triggered by rule-based conditions, or inferred by a machine-learning model to produce different orchestral realizations of the same underlying harmonic function.



```

<> JSON

{
  "layer": "harmony",
  "preserve": [
    "harmonic_progression",
    "harmonic_function",
    "cadential_structure"
  ],
  "modules": {
    "sustained_harmony": {
      "enabled": true,
      "weight": 0.9
    },
    "common_tone_retention": {
      "enabled": true,
      "weight": 0.8
    },
    "contrary_motion": {
      "enabled": true,
      "weight": 0.5
    },
    "incomplete_chords": {
      "enabled": false
    }
  }
}

```

LAYER 3 – Bassline

The Bassline layer transforms the structural bass of the input material into an orchestral bass foundation. It first extracts the structural bassline underlying the harmonic progression. Based on this, it generates a bass part that establishes harmonic support while reinforcing the rhythmic and tonal stability of the orchestral texture. The bassline is then assigned to the instruments of the bass layer—e.g., cello and double bass—and optimized for register, balance, and interaction with the Harmony layer. The realization may employ orchestration principles such as simple harmonic bass, octave doubling, pedal point, ostinato, arpeggiated bass, stepwise bass motion, or rhythmic unison, depending on the desired orchestral character. The result is an orchestral bass texture that preserves the structural harmonic foundation while transforming the pianistic bass into a stable, supportive orchestral foundation.

BASSLINE LAYER – RULE STEPS

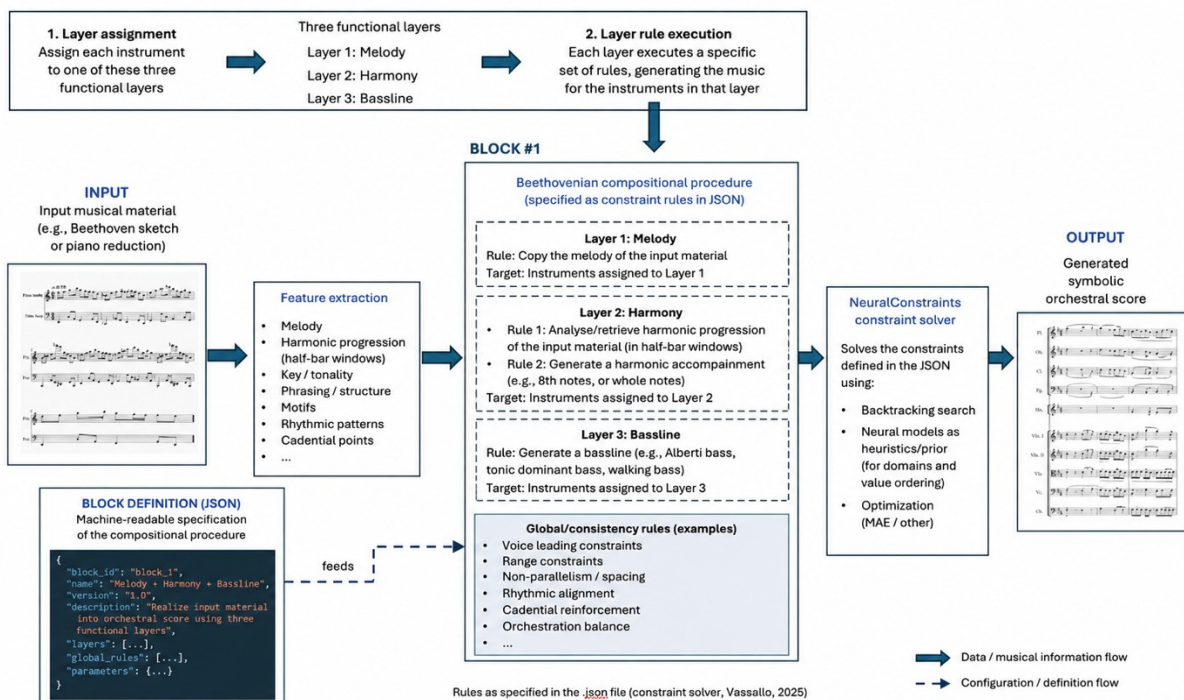
From symbolic input to orchestral bass foundation

Step	Rule	Example
1. Extract	Extract the structural bass function underlying the harmonic progression.	Piano bass → structural harmonic bass.
2. Preserve	Preserve the harmonic foundation, tonal centre, cadential function, and structural bass motion.	Root motion, cadential arrival, pedal function.
3. Generate	Generate an orchestral bass foundation that supports harmony and rhythm.	Continuous orchestral bass replacing pianistic figuration.
4. Assign & Optimize	Assign bass functions to cello, double bass (and optionally bassoon/tuba) and optimize register, balance, and interaction with the Harmony layer.	Octave doubling, register distribution, balance.
5. Realize	Produce the orchestral bass texture using one or more orchestration strategies.	Orchestration strategies for the bassline: • Simple harmonic bass • Octave doubling • Pedal point / bass • Ostinato • Arpeggiated bass • Stepwise bass • Rhythmic unison bass

Outlook and next steps

The implementation of Block #1 establishes the underlying architecture of the Beethoven Blocks framework. Once this foundation has been validated, additional Beethoven Blocks can be introduced simply by defining new JSON-based compositional procedures, without requiring modifications to the underlying NeuralConstraints engine. Importantly, the JSON specification does not encode an orchestral score itself. Rather, it encodes the compositional procedure that transforms arbitrary symbolic musical material into such a score. In this way, Beethovenian compositional knowledge is represented explicitly as machine-readable realization procedures, while the NeuralConstraints engine remains a generic execution framework responsible for realizing these procedures.

The next project milestone is the implementation and demonstration of the first working Beethoven Block. The objective is for Tuur to present a proof-of-concept implementation of Block #1 during Kit Armstrong's visit to Ghent on 2 September. This demonstration will validate the proposed block-based realization framework and provide a foundation for evaluating the architecture, discussing additional Beethoven Blocks, and planning subsequent development.



References

Vassallo, J. S., Sandred, Ö., & Vincenot, J. (2025). *NeuralConstraints: Integrating a neural generative model with constraint-based composition*. *Frontiers in Computer Science*, 7, Article 1543074.

Appendix

The meeting elaborated on an email of Kit (21/7) replying to the question of whether he could provide some examples of block types. The email is the following:

“Dear Pieter-Jan, sorry for the delay!

Here are some, I think, good examples of block

types: <https://drive.google.com/drive/folders/1p32uNtRZoUbcdSgaH4dnZFiDP8jtAxMY> . (I took these from Beethoven Op.37)

1a,1b: 8-bar melody. Start with a melody -> harmonise -> orchestrate (basic example: clarinet + violin = melody; viola, cello, bass = bass line; everyone else = harmony.) I guess this is the most difficult one, since it involves knowing about good composition.

2: one harmony (in this case G7), different inversions and different instruments on each beat. Can be any number of bars. Piano soloist plays on top of it.

3: like 2, but more complex: stereotypical harmonic sequence + stereotypical rhythmic texture. Can be any number of bars.

4: same as 3, but add motifs (from the predetermined "motif toolbox") where they fit in the harmony.

5: series of motifs in unison. Multiple of 4 bars.

6: call-and-response: choose 1 harmony + rhythm of motif, then alternate with the same number of blank bars. (in this case: 2 iterations, with harmony = [Cm, Fm inv. 2], rhythm = | cq.s|c)

I hope this helps. I would be happy to talk together to further develop/clarify. Good times for me would be 9am on 23 July, or 27-30 July.

Best regards,

Kit

„

Via this email references were made to Figures:

1a.png

The image displays two pages of a musical score, numbered 16 and 17. The score is written for a large ensemble, including Clarinet (Cl.), Saxophone (Sax.), Violin (Vl.), Viola (Vla.), Cello (Vc.), Double Bass (Cb.), Piano (Pfte.), Flute (Fl.), Oboe (Ob.), and Bassoon (Fg.). The notation is in standard musical staff format with various dynamics and performance markings. On page 16, there is a green bracket highlighting a section of the score. On page 17, there is a green bracket highlighting a section of the score. The score includes dynamic markings such as 'sempre stacc.' and 'cresc.' and performance instructions like 'TUTTI' and 'SOLO'.

1b.png

80

Fl.
Cl.
(B)
Fg.
Cor.
(Es)
Vl.
Vla.
Vc.
Cb.

80

SOLO

2.png

SOLO

Fl.
Ob.
Cl.
(B)
Fg.
Cor.
(Es)
Vl.
Vla.
Vc.
Cb.

SOLO

SOLO

3.png

3.png

Handwritten musical score for strings and piano. The score is in B-flat major (two flats) and 4/4 time. It features five staves: Violin I (Vl.), Violin II (Vla.), Viola (Vc.), Cello (Cb.), and Piano (Pfte.). A green bracket on the left side groups the string staves (Vl., Vla., Vc., Cb.). The piano part (Pfte.) is written in a grand staff (treble and bass clef). The score includes dynamic markings: *p* (piano) and *cresc.* (crescendo). The string parts are mostly sustained notes, while the piano part features a complex, flowing melody with trills and grace notes.

4.png

4.png

Handwritten musical score for woodwinds, strings, and piano. The score is in B-flat major (two flats) and 4/4 time. It features seven staves: Flute (Fl.), Clarinet in B-flat (Cl. (B)), Bassoon (Fg.), Violin I (Vl.), Violin II (Vla.), Viola (Vc.), Cello (Cb.), and Piano (Pfte.). A green bracket on the left side groups the string staves (Vl., Vla., Vc., Cb.). The woodwind parts (Fl., Cl. (B), Fg.) are mostly sustained notes, while the piano part (Pfte.) is written in a grand staff (treble and bass clef). The score includes dynamic markings: *p* (piano). The string parts are mostly sustained notes, while the piano part features a complex, flowing melody with trills and grace notes.

5.png

6.png

[illegible]

Ob.

Cl. (B)

Fg.

Cor. (Es)

Tbe. (C)

Timp.

SOLO

60

Pfte.

SOLO